

1 Basic MNIST CNN

This section discusses ways of building a convolutional neural network for the MNIST dataset.

1.1 Plain CNN

This is the result from training the most basic CNN. The data was split into training and test data. The curve represents a normal behaviour of training loss, however it must be stated, that no validation dataset was used. Due to this, getting an overview of how the model performs on unseen data can be done only through looking at the test results. During the test the model achieved a raw accuracy of 98.74%.

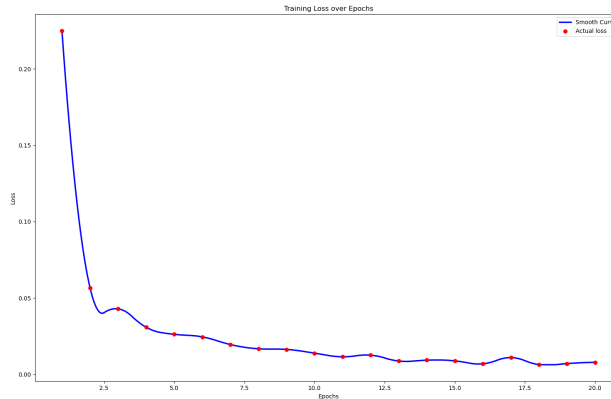


Figure 1: Plain CNN loss curve, Test Accuracy 98.74%

1.2 Train data augmentation and Learning Rate Scheduler

These are the results from training the CNN with augmented train data, a learn rate scheduler and a validation set (variations with and without certain elements), while keeping the same architecture.

The following graph presents the loss curve of a model that receives slightly augmented (affine transformations) train data and uses a learning rate scheduler during training. As it is visible after epoch 13 the loss hits a plateau, likely caused by the scheduler. It is visible that after this epoch no meaningful gains are made, suggesting that the learning rate has been reduced to almost 0. During the first run the model achieved 98.31%. After a second run where some implementation improvements were made, the performance jumped to 99.33%.

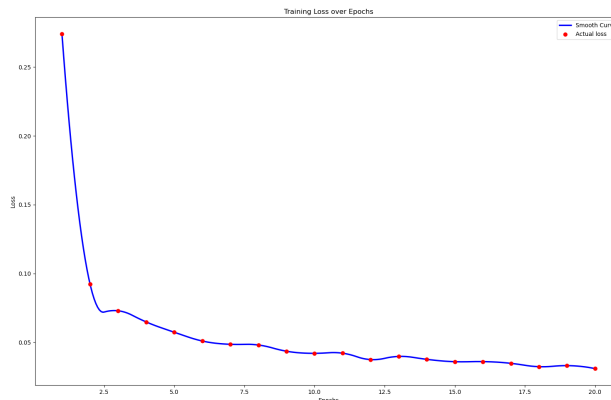


Figure 2: Second run loss curve, Test Accuracy 99.33%

Regarding the suggestions made, a couple more simulations were made. The results of these simulations not only did not support the statements made in the previous paragraph, but also gave a clearer picture of what was happening during training. Using only train data augmentation two models with the same architecture were made, but with different implementations of augmentation. The first model achieved an accuracy of 97.94%, where as the second model's precision went up by almost 1.5%. On the other side the usage of a learning rate scheduler was not as beneficial as suggested previously. As it can be seen from the results, no real improvement was made and the plateauing of the loss cannot be solely attributed to its introduction.

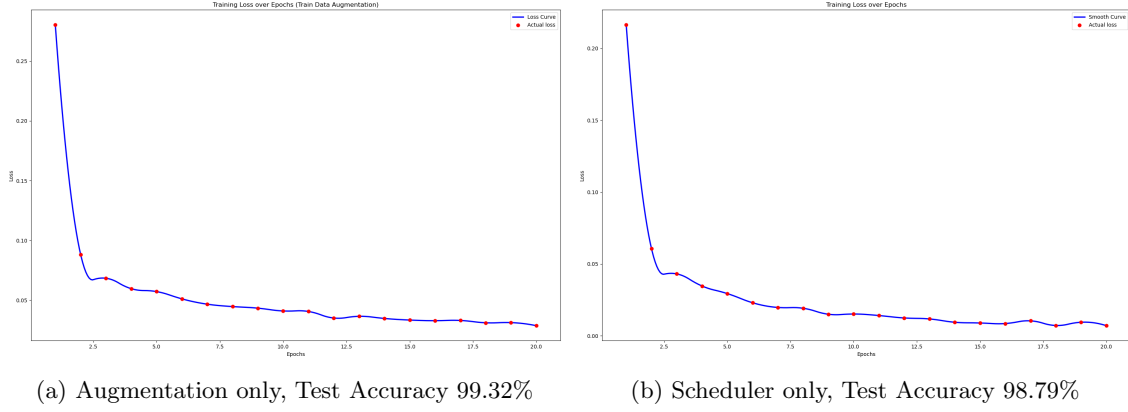


Figure 3

While keeping the same architecture, the train data was split into a train set and a validation set selecting the best settings from previous experiments. As we can see in the graph the validation accuracy starts plateauing after epoch 13, confirming the findings and suggestions from previous experiments. Although it must be noted that the difference between validation loss and training loss is significant and could hint at moderate overfitting.

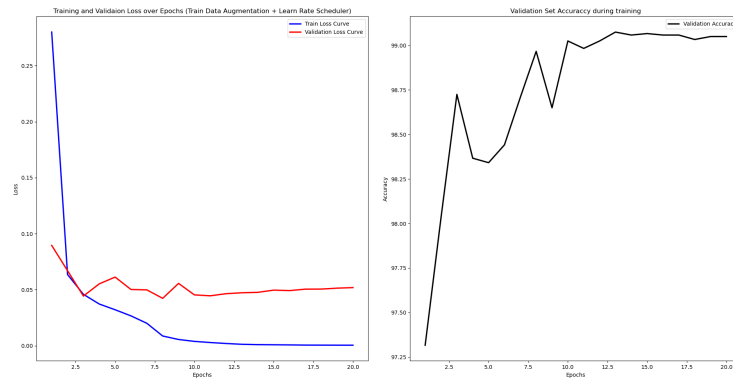


Figure 4: Introduction of Validation, Test Accuracy 99.28%

1.3 Introduction of Batchnorm and Dropout

In order to reduce the gap between the train loss and validation loss, certain changes were made to the CNN. These changes mainly focused on hyperparameters like number of features extracted during convolution, padding added to the images, and kernel size.

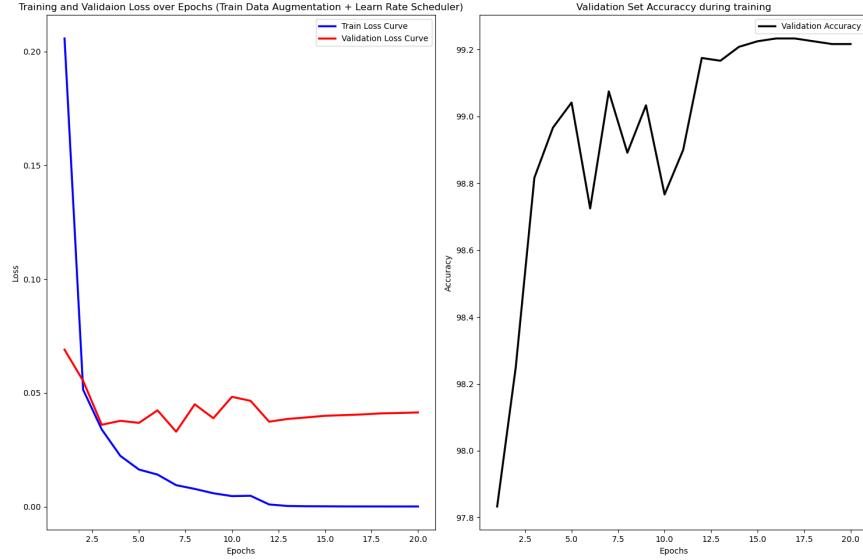


Figure 5: Hyperparameter tuning, Test Accuracy 99.32%

Since the gap between the losses did not reduce significantly other changes to the architecture of the CNN were made. The following graph displays the performance of the model when Batchnorm and Dropout for the convolution layers are introduced. While these modifications did not bring any accuracy improvement on test set, the gap between train loss and validation loss started to close.

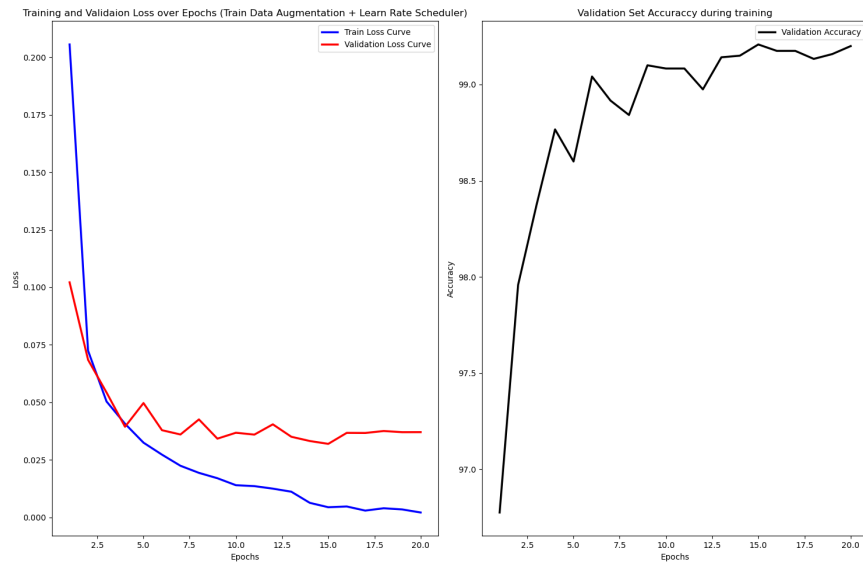


Figure 6: Batchnorm & Dropout impact, Test Accuracy 99.25%

Based on the results from the previous run where batchnorm and dropout for convolutional layers were introduced, in this run dropout was also introduced for the deep layers. This introduction proves significant as for the first time since validation introduction, the losses start to converge, with training loss hitting a plateau higher than before.

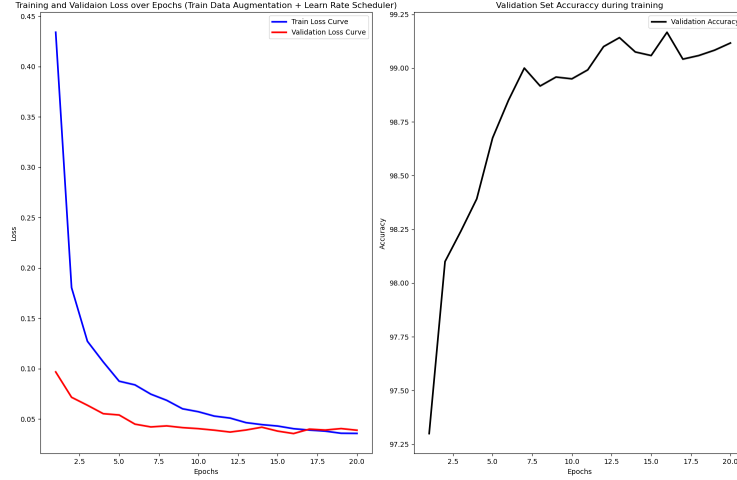


Figure 7: Dropout deep layers, Test Accuracy 99.37%

1.4 Training over longer time

After having taken all these steps and undergoing all the aforementioned changes, there still was one last parameter that had stayed fixed the entire time, namely the epochs. In this final run for the basic MNIST CNN the number of epochs was increased to 30. Although from the validation charts of previous models the expectation was for this change to improve test scores, the results disagree.

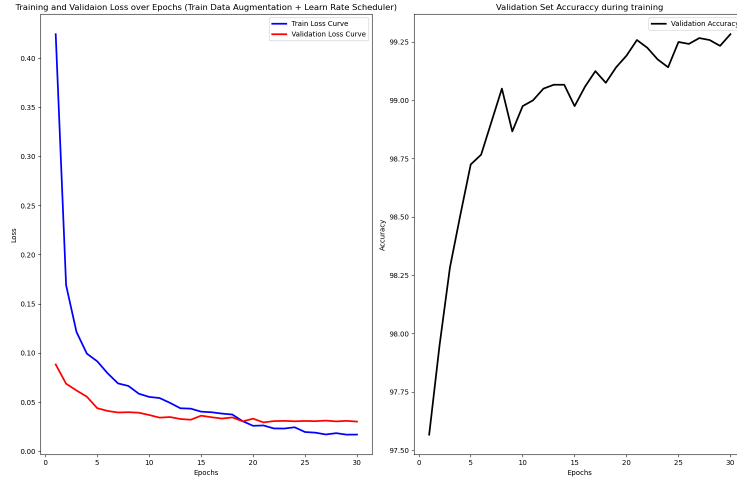


Figure 8: Increased no. epochs, Test Accuracy 99.37%

2 Advanced MNIST CNN

This subsection builds on the CNN discussed previously, by adding another output head, that predicts whether a number is odd or even.

2.1 Hierarchical MNIST CNN

The approach models the Odd/Even-output head as a single perceptron that outputs a probability distribution over odd and even. This distribution directly shapes the digit prediction, because instead of letting the

digit head predict freely across all 10 digits, we use those probabilities to build a soft mask that suppresses impossible digits. Such model allows for the use of a single CrossEntropyLoss (that of the digit). Below the charts show the results of two models, who use the same approach but use slightly different loss calculations.

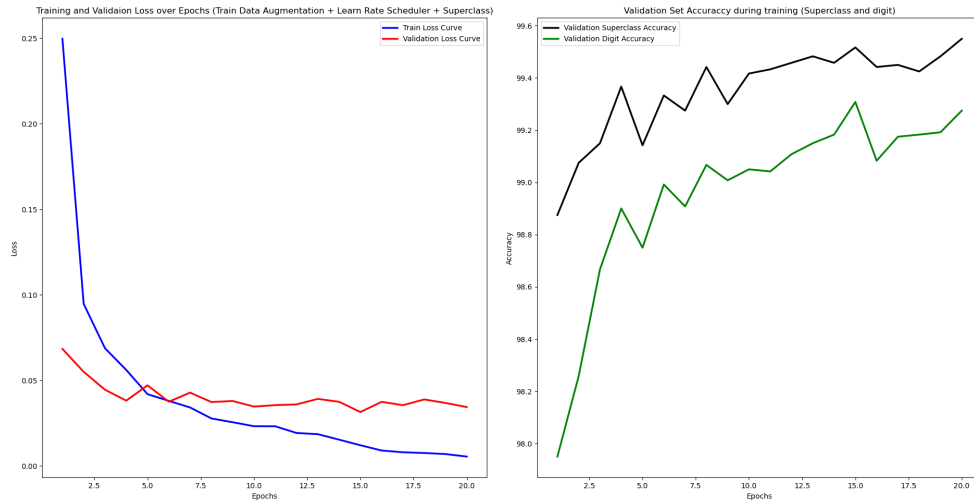


Figure 9: Loss=0.9*digit+0.1*super, Superclass Accuracy 99.67%, Digit Accuracy 99.290%

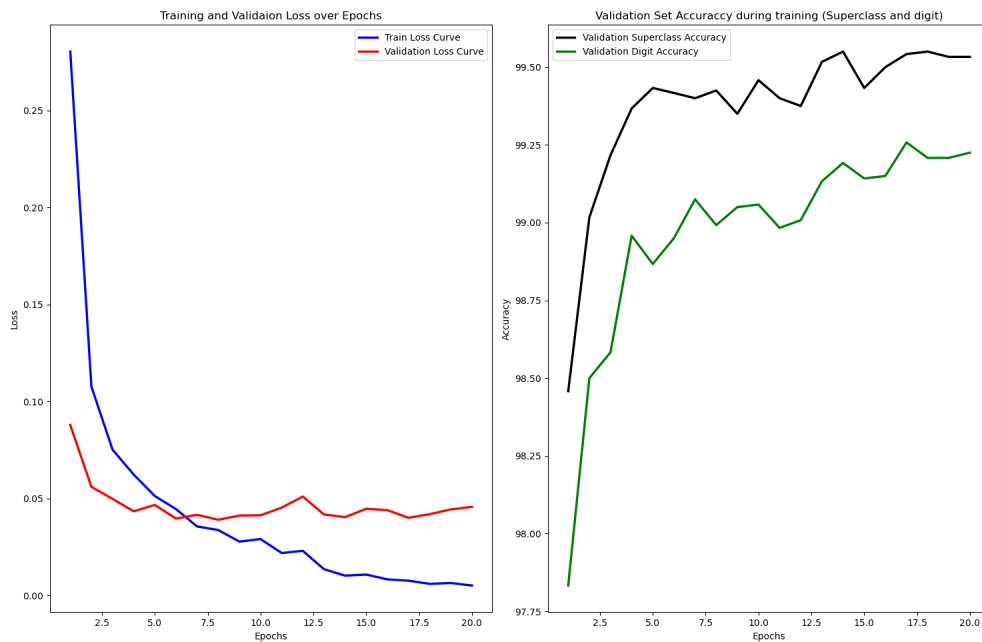


Figure 10: Loss=digit, Superclass Accuracy 99.67%, Digit Accuracy 99.260%

2.2 Train Expert MNIST CNN

In contrast to the Hierarchical CNN the Train Expert CNN uses two output heads to predict the digit based on the output of the additional superclass predicting head. The approach allows for a stricter way of measuring loss. However, the drawback is that a wrong prediction of superclass leads to a wrong prediction of the digit.

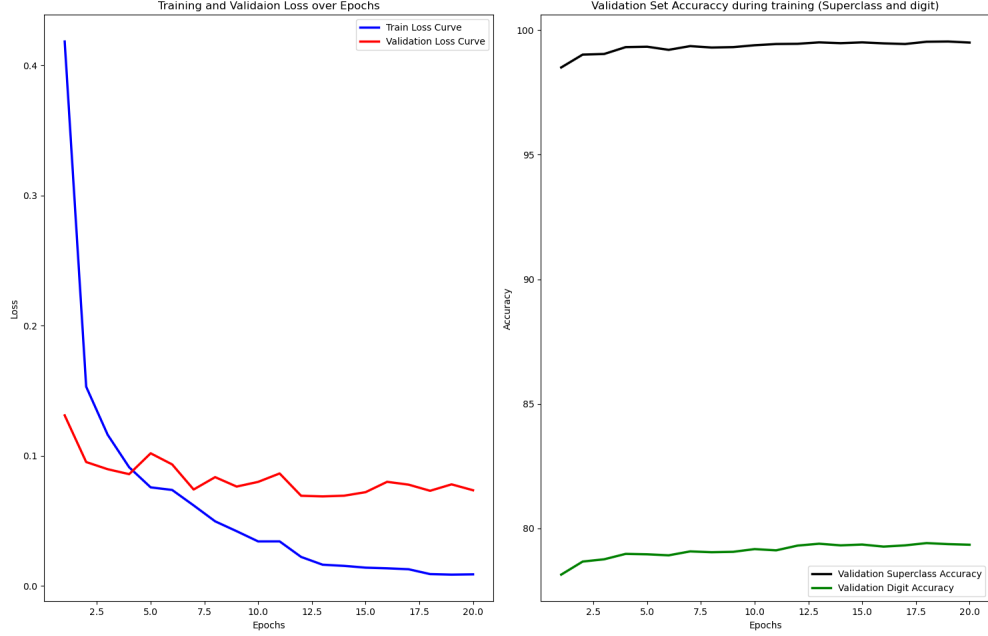


Figure 11: Superclass Accuracy 99.61%, Digit Accuracy 80.00%

3 Closing Remarks

In conclusion designing, training and optimizing a Classifier takes more than just building a textbook Convolutional Neural Network. While this is still a very simple CNN, improvements like data augmentation and learning rate scheduler add more to the robustness and consistency of the model. Furthermore techniques like validation provide important information about the performance of the model during training and help better understand the strengths and vulnerabilities. Regarding the Hierarchical and Train Expert models, it can be said that one can classify the data with respect to more than one feature, however choosing the right approach heavily impacts performance for the better of it.